

Guia de Consumo de APIs REST

Aula 04 — REST: Revisão · Postman, cURL e Swagger, do `git clone` ao primeiro 201

Professor Thalles Noce · duas APIs para praticar: **Café Newton** (a correta) e **PetHouse** (a que tem 12 erros)

O que você vai fazer nesta aula

1. Baixar o repositório e rodar duas APIs REST na sua máquina.
2. Consumi-las por três caminhos diferentes: **Swagger** (nada para instalar), **Postman** e **cURL**.
3. Conferir na API **Café Newton** como uma resposta REST correta se parece (200, 201 + `Location`, 400, 404, 204, 304).
4. Auditar a API **PetHouse** e encontrar os **12 erros de design** — um por endpoint.

1. Pré-requisitos

Ferramenta	Como conferir	Onde obter / observação
Git	<code>git --version</code>	<code>git-scm.com/downloads</code> — no Windows já instala o <i>Git Bash</i> , útil para os scripts <code>.sh</code> .
.NET 6 SDK (ou superior)	<code>dotnet --version</code>	<code>dotnet.microsoft.com/download</code> . Os projetos têm <code>RollForward</code> ligado: se você já tem .NET 8, 9 ou 10, eles rodam do mesmo jeito.
cURL	<code>curl --version</code>	Já vem no Windows 10/11, no macOS e no Linux. Nada a instalar.
Postman	—	<code>postman.com/downloads</code> (opcional: dá para fazer tudo pelo Swagger e pelo cURL).

2. Baixar e rodar as APIs

2.1 Clonar o repositório

```
git clone https://github.com/thallesnoce/newton-aaw-26.2.git
cd newton-aaw-26.2
```

Se você já clonou o repositório em outra aula, basta atualizar:

```
cd newton-aaw-26.2
git pull
```

2.2 Subir a API correta — Café Newton (porta 5301)

```
cd "aulas/aula04-rest/src/CafeteriaApi"
dotnet run
```

A primeira execução baixa os pacotes e compila (leva alguns segundos). Quando aparecer `Now listening on: http://localhost:5301`, a API está no ar. **Deixe este terminal aberto** — fechar ou apertar `Ctrl+C` derruba a API.

2.3 Subir a API para auditoria — PetHouse (porta 5302)

Abra um **segundo terminal** (o primeiro está ocupado rodando a Cafeteria):

```
cd "aulas/aula04-rest/src/PetShopApi"
dotnet run
```

Confirme que as duas subiram

Abra no navegador: `http://localhost:5301/swagger` e `http://localhost:5302/swagger`. Você deve ver duas páginas de documentação diferentes. A raiz de cada API (`http://localhost:5301/`) redireciona para o Swagger.

Os dados ficam só na memória

As duas APIs guardam tudo em listas na memória do processo. Se você bagunçar os dados (apagar, criar coisas demais), **pare com `Ctrl+C` e rode `dotnet run` de novo**: tudo volta ao estado inicial. Isso também significa que o que você criar hoje some quando o computador desligar — e é justamente esse o gancho da próxima aula.

3. Anatomia de uma requisição HTTP

Toda chamada que você vai fazer, em qualquer ferramenta, tem sempre as mesmas cinco partes. Saber apontar cada uma é metade da prática de hoje.

```
POST http://localhost:5301/api/v1/cafes?page=1&size=20
├── método HTTP: o QUE FAZER
├── host e porta
└── ┬── filtros e paginação
    └── recurso (substantivo, plural)
        └── versão do contrato

Content-Type: application/json    ← headers da requisição

{ "nome": "Espresso da Aula" }   ← corpo, só em POST/PUT/PATCH

--- resposta ---
HTTP/1.1 201 Created             ← status code: o que mais importa
Location: /api/v1/cafes/241      ← headers da resposta
{ "id": 241, "nome": "..." }   ← corpo da resposta
```

Os status codes que você vai ver hoje

Código	Nome	Quando aparece
200	OK	Leitura ou atualização deu certo, e o corpo traz o resultado.
201	Created	Um recurso novo passou a existir. Sempre acompanhado do header <code>Location</code>.
204	No Content	Deu certo e não há nada para devolver (o caso típico do <code>DELETE</code>).
304	Not Modified	O cliente já tem esta versão em cache; a resposta vem sem corpo.
400	Bad Request	O cliente mandou algo inválido (faltou um campo, um número fora da faixa).
404	Not Found	O recurso pedido não existe.
405	Method Not Allowed	A URI existe, mas não aceita esse método (ex.: <code>GET</code> numa rota só de <code>POST</code>).
500	Internal Server Error	O servidor quebrou. Culpa nossa, não do cliente.

Regra de ouro da aula

A **URI** identifica **o QUE** (o recurso, sempre um substantivo no plural). O **método HTTP** diz **o QUE FAZER**. Se você viu um verbo dentro da URL (`/getPets` , `/deletarPet`), alguma coisa está errada.

4. Consumindo pelo Swagger UI (sem instalar nada)

Cada API já vem com o Swagger embutido — é a forma mais rápida de começar.

1. Abra `http://localhost:5301/swagger`.
2. Clique em um endpoint para expandir. O Swagger mostra os parâmetros, o formato do corpo (*Schema*) e os status codes possíveis.
3. Clique em **Try it out**, preencha os campos e clique em **Execute**.
4. Leia a resposta prestando atenção em **três coisas**:
 - **Server response / Code** — o status code (200? 201? 404?);
 - **Response body** — o corpo;
 - **Response headers** — aqui aparecem `Location`, `ETag` e `Cache-Control`. *Boa parte dos erros da aula de hoje só é visível nos headers.*

Truque: o Swagger escreve o cURL para você

Depois do *Execute*, o Swagger mostra o campo **Curl** com o comando exato daquela chamada. É a melhor forma de aprender a sintaxe do cURL sem decorar nada: monte a requisição no Swagger, copie o comando e cole no terminal.

Roteiro rápido no Swagger (API Café Newton)

Faça isto	Espere ver
GET <code>/api/v1/cafes</code> com <code>size=5</code>	200 e um corpo com <code>page</code> , <code>size</code> , <code>total</code> e <code>items</code>
GET <code>/api/v1/cafes/1</code>	200 e, nos <i>response headers</i> , <code>ETag</code> e <code>Cache-Control</code>
POST <code>/api/v1/cafes</code> com um corpo válido	201 e o header <code>Location</code> com a URL do café criado
POST <code>/api/v1/cafes</code> sem o campo <code>nome</code>	400 e a lista de erros de validação
DELETE <code>/api/v1/cafes/5</code>	204 na primeira, 404 na segunda duas vezes

5. Consumindo pelo Postman

5.1 Importar as coleções prontas

As duas coleções estão em `aulas/aula04-rest/src/testes/`:

- `CafeteriaApi.postman_collection.json` — 20 requisições, agrupadas em "caminho feliz", "casos que ensinam" e "pedidos e itens";
- `PetShopApi.postman_collection.json` — os 12 endpoints a auditar, na ordem, com uma pergunta em cada descrição.

No Postman: **Import** → arraste os dois arquivos → **Import**. Cada coleção já traz a variável `baseUrl` apontando para a porta certa (`{{baseUrl}}` vira `http://localhost:5301` ou `5302`). Se precisar mudar a porta, edite em *Collection* → *Variables* — muda em todas as requisições de uma vez.

5.2 O que olhar em cada resposta

Onde	O que ler
Barra superior da resposta	Status (ex.: <code>201 Created</code>), Time e Size . O tamanho denuncia o endpoint sem paginação.
Aba Body	O JSON. Use a visão <i>Pretty</i> .
Aba Headers (da resposta)	<code>Location</code> , <code>ETag</code> , <code>Cache-Control</code> , <code>Content-Type</code> .

5.3 Montando uma requisição do zero

1. Escolha o método no seletor à esquerda da URL (`GET` , `POST` , `PUT` , `PATCH` , `DELETE`).
2. Digite a URL: `http://localhost:5301/api/v1/cafes` .
3. Para enviar um corpo: aba **Body** → **raw** → selecione **JSON** no seletor à direita (isso já configura o header `Content-Type: application/json`).
4. Cole o JSON e clique em **Send**.

```
{
  "nome": "Espresso da Aula",
  "origem": "Sul de Minas",
  "torra": "media",
  "preco": 9.5,
  "disponivel": true
}
```

Erro clássico no Postman

Enviar o corpo como *raw / Text* em vez de *raw / JSON*. A API responde `415 Unsupported Media Type` ou `400` . Confira sempre o seletor ao lado de "raw".

6. Consumindo por cURL

O cURL é o menor denominador comum: está em todo servidor, em toda pipeline de CI, em toda documentação de API. Vale a pena conhecer meia dúzia de flags.

Flag	Para que serve
<code>-i</code>	Mostra os headers da resposta junto com o corpo. <i>Use sempre nesta aula.</i>
<code>-X MÉTODO</code>	Define o método (<code>-X POST</code> , <code>-X DELETE</code> ...). Sem ele, o cURL faz <code>GET</code> .
<code>-H "..."</code>	Adiciona um header (<code>-H "Content-Type: application/json"</code>).
<code>-d '{...}'</code>	Envia o corpo. Também pode ler de arquivo: <code>-d "@corpo.json"</code> .
<code>-s</code>	Silencioso: esconde a barra de progresso.
<code>-v</code>	Verboso: mostra a requisição inteira, header por header. Ótimo para depurar.
<code>-o arquivo</code>	Grava o corpo num arquivo (<code>-o /dev/null</code> descarta).
<code>-w "%{http_code}"</code>	Imprime só o que você pedir: status, tamanho, tempo.

Um exemplo por verbo (API Café Newton)

```
# GET – ler a coleção paginada
curl -i "http://localhost:5301/api/v1/cafes?page=1&size=3"

# GET – ler um recurso (repare no ETag da resposta)
curl -i http://localhost:5301/api/v1/cafes/1

# POST – criar (procure o header Location na resposta: 201 Created)
curl -i -X POST http://localhost:5301/api/v1/cafes \
  -H "Content-Type: application/json" \
  -d '{"nome":"Espresso","origem":"Minas","torra":"media","preco":9.5}'

# PUT – substituir o recurso inteiro (idempotente: repita à vontade)
curl -i -X PUT http://localhost:5301/api/v1/cafes/2 \
  -H "Content-Type: application/json" \
  -d '{"nome":"Coador","origem":"Minas","torra":"clara","preco":11}'

# PATCH – alterar só um campo
curl -i -X PATCH http://localhost:5301/api/v1/cafes/2 \
  -H "Content-Type: application/json" -d '{"preco":13.75}'

# DELETE – remover (204 na primeira vez, 404 na segunda)
curl -i -X DELETE http://localhost:5301/api/v1/cafes/3
```

Medindo a resposta (útil no endpoint sem paginação)

```
curl -s -o /dev/null \  
  -w "%{http_code} · %{size_download} bytes · %{time_total}s\n" \  
  http://localhost:5302/api/v1/consultas
```

Windows: duas armadilhas

1. curl no PowerShell não é o cURL. No Windows PowerShell 5.1, `curl` é apelido de `Invoke-WebRequest`, que não entende `-X`, `-H` nem `-d`. Escreva sempre `curl.exe`.

2. Aspas. O PowerShell "come" as aspas duplas do JSON e quebra o argumento nos espaços. A forma que funciona em qualquer versão é gravar o corpo num arquivo e usar `-d "@corpo.json"`:

```
$corpo = '{ "nome": "Espresso", "origem": "Minas", "preco": 9.5 }'  
Set-Content corpo.json $corpo  
curl.exe -i -X POST http://localhost:5301/api/v1/cafes \  
  -H "Content-Type: application/json" -d "@corpo.json"
```

Scripts prontos

Todos os comandos acima já estão empacotados em `src/testes/`:

```
# Git Bash, WSL, macOS ou Linux  
bash curl-cafeteria.sh  
bash curl-petshop.sh  
  
# PowerShell (Windows)  
powershell -ExecutionPolicy Bypass -File .\curl-cafeteria.ps1  
powershell -ExecutionPolicy Bypass -File .\curl-petshop.ps1
```

Prefere não instalar nem o Postman nem usar o terminal? Os arquivos `cafeteria.http` e `petshop.http` rodam direto no VS Code (extensão **REST Client**), no Visual Studio 2022 e no Rider: abra o arquivo e clique em *Send Request* acima de cada bloco.

7. Parte 1 — Verificação na API Café Newton

Antes de caçar erros, é preciso saber como o certo se parece. Rode os sete casos abaixo (na ferramenta que você preferir) e anote a resposta de cada um.

#	Requisição	Esperado	O que observar
1	<code>GET</code> <code>/api/v1/cafes?page=1&size=5</code>	200	O envelope <code>page/size/total/items</code> . São 240 cafés — mas só 5 viajam.
2	<code>GET</code> <code>/api/v1/cafes?size=100000</code>	400	Quem define o teto é o servidor (máx. 100), não o cliente.
3	<code>GET</code> <code>/api/v1/cafes/1</code>	200	Headers <code>ETag</code> e <code>Cache-Control</code> . Anote o ETag.
4	<code>GET</code> <code>/api/v1/cafes/1</code> com <code>If-None-Match: "cafe-1-v1"</code>	304	Resposta sem corpo : o cliente já tinha essa versão.
5	<code>POST</code> <code>/api/v1/cafes</code> (corpo válido)	201	O header <code>Location</code> traz a URI do café criado.
6	<code>POST</code> <code>/api/v1/cafes</code> sem <code>nome</code>	400	A lista de erros de validação, campo por campo.
7	<code>DELETE</code> <code>/api/v1/cafes/{id}</code> duas vezes	204 e 404	Por que a segunda chamada dá 404 — e por que isso <i>ainda</i> é idempotente?

Explore também o aninhamento correto, de um nível só:

<code>GET</code>	<code>/api/v1/pedidos/1/itens</code>	
<code>POST</code>	<code>/api/v1/pedidos/1/itens</code>	<code>{ "cafeId": 5, "quantidade": 2 }</code>
<code>PATCH</code>	<code>/api/v1/pedidos/1</code>	<code>{ "status": "entregue" }</code>

Repare que **não existe** `/pedidos/1/entregar`. Mudar o status é alterar um campo do recurso — a ação virou dado, e o recurso continua sendo um substantivo.

8. Parte 2 — Missão na API PetHouse: os 12 erros

A API funciona. E é esse o problema.

Nenhuma requisição da PetHouse quebra: todas respondem. Foi por isso que ela passou na revisão de código. Ainda assim, **cada endpoint viola uma regra ou diretriz REST** vista em aula. São doze endpoints e doze erros diferentes.

Chame cada endpoint (Postman, cURL, Swagger ou `petshop.http`) e preencha a tabela do entregável. A coluna "o que observar" diz onde olhar — **não** diz qual é o erro.

#	Endpoint	O que observar
01	<code>POST</code> <code>/api/v1/getPets</code>	Leia a URI em voz alta. E por que POST, se só está lendo?
02	<code>GET</code> <code>/api/v1/deletarPet?id=7</code>	Consulte o pet antes e depois. O que um GET acabou de fazer? Essa URL cola na barra do navegador.
03	<code>GET</code> <code>/api/v1/pet/{id}</code>	Compare esta rota com a do endpoint 06. Qual é o padrão da casa?
04	<code>GET</code> <code>/api/v1/banhosTosa</code> <code>GET</code> <code>/api/v1/tutores_vip</code>	A grafia das duas rotas (e a do endpoint 12). E: "vip" é um recurso ou um filtro?
05	<code>POST</code> <code>/api/v1/pets</code>	Qual status voltou? Existe header <code>Location</code> ? Como o cliente descobre a URL do que criou?
06	<code>GET</code> <code>/api/v1/pets/999999</code>	O pet não existe. Que status code a API devolveu — e o que o corpo diz?
07	<code>GET</code> <code>/api/pets</code>	Compare a URI e os nomes dos campos com os do endpoint 01. O que acontece com o app de 2024?
08	<code>GET</code> <code>/api/v1/petshops/1/clientes/5/pets/9/consultas/12/exames/6</code>	Quanto ids o cliente precisa conhecer? Troque os três primeiros por 99 e compare a resposta.
09	<code>GET</code> <code>/api/v1/consultas</code>	O tamanho e o tempo da resposta (rodapé do Postman). E se a clínica tiver 60.000 consultas?
10	<code>PUT</code> <code>/api/v1/pets/12/vacinas</code>	Chame três vezes com o corpo idêntico e conte as vacinas a cada chamada.
11	<code>POST</code> <code>/api/v1/sessao</code> <code>GET</code> <code>/api/v1/meus-pets</code>	Faça login como "Ana" e veja seus pets. Peça a um colega que faça login como "Bruno". Consulte de novo — sem refazer seu login.
12	<code>GET</code> <code>/api/v1/tabela-de-precos</code>	Os headers da resposta. Este dado é reajustado uma vez por ano .

Entregável

Uma tabela (documento, planilha ou a coleção Postman exportada com comentários) com uma linha por endpoint:

#	O que eu chamei	O que a resposta mostrou	Regra REST violada	Como eu redesenharia
01				
02				
...				

Dica: a API Café Newton faz certo tudo o que a PetHouse faz errado. Quando travar em um endpoint, procure o equivalente na porta 5301 e compare as duas respostas lado a lado.

9. Troubleshooting

Sintoma	O que fazer
<code>dotnet: command not found</code>	O SDK do .NET não está instalado ou não está no PATH. Instale o .NET 6 SDK (ou superior) e abra um terminal novo.
<code>Failed to bind to address ... address already in use</code>	A porta já está ocupada — provavelmente por outra instância da mesma API. Feche o terminal antigo, ou rode em outra porta: <code>dotnet run --urls http://localhost:5399</code> .
O navegador mostra "não foi possível acessar o site"	A API não está no ar. Volte ao terminal e confira se apareceu <code>Now listening on: http://localhost:5301</code> . Se você apertou <code>Ctrl+C</code> , rode <code>dotnet run</code> de novo.
Erro de certificado / <code>https</code> não abre	As duas APIs rodam só em http de propósito, para evitar isso. Confira se a URL não começa com <code>https://</code> .
No PowerShell, o <code>curl</code> reclama de <code>-X</code> ou de <code>-d</code>	Você chamou o alias <code>Invoke-WebRequest</code> . Escreva <code>curl.exe</code> e passe o corpo por arquivo: <code>-d "@corpo.json"</code> .
<code>400 Bad Request</code> num POST que parece certo	Quase sempre é o JSON: vírgula sobrando, aspas simples em vez de duplas, ou o header <code>Content-Type: application/json</code> ausente. No Postman, use <i>Body</i> → <i>raw</i> → <i>JSON</i> .
<code>415 Unsupported Media Type</code>	Faltou o header <code>Content-Type: application/json</code> .
<code>405 Method Not Allowed</code>	A URI existe, mas não aceita esse método. (Na PetHouse, isso acontecer é parte do exercício — anote.)
Os dados ficaram bagunçados	<code>Ctrl+C</code> no terminal da API e <code>dotnet run</code> de novo: os dados voltam ao estado inicial.
O Swagger abre em branco	Confira a URL: <code>http://localhost:5301/swagger</code> (com a porta certa) e recarregue com <code>Ctrl+F5</code> .

10. Resumo dos endereços

O quê	Onde
Repositório	<code>github.com/thallesnoce/newton-aaw-26.2</code>
Pasta da aula	<code>aulas/aula04-rest/</code>
API correta (Café Newton)	<code>src/CafeteriaApi</code> → <code>http://localhost:5301/swagger</code>
API para auditoria (PetHouse)	<code>src/PetShopApi</code> → <code>http://localhost:5302/swagger</code>
Coleções Postman, <code>.http</code> e scripts <code>curl</code>	<code>src/testes/</code>

Uma boa API é aquela que o próximo dev usa sem precisar te perguntar nada. Ao terminar a prática, releia os 12 erros da PetHouse com essa frase na cabeça: cada um deles é uma pergunta que o próximo dev teria de fazer para você.